

To get Internet connection inside, use the following command:

```
#sudo cp /etc/resolv.conf /var/chroot/etc/resolv.conf
```

After that, you are ready to enter your environment.

```
#chroot my_deb/ /bin/bash
```

Here, you get a whole basic operating system inside your *chroot*. But it differs from your main system by mount point, because it only uses the mount property as the isolator. It has the same hostname, IP address and process running as in the main system. That's why it is much less secure (this is even mentioned in the man page of *chroot*), and any running process can still harm your computer by killing your tasks or affecting network based services.

 **Note:** To run graphical applications inside *chroot*, open x server by running the following command on the main system:

```
#xhost +
and on chroot system
#export DISPLAY=:0.0
```

On *systemd* based systems, *chrooting* is pretty straightforward. It's needed to define the root directory on the processes unit file only.

```
[Unit]
Description=my_chroot_Service
[Service]
RootDirectory=/chroot/foobar
ExecStartPre=/usr/local/bin/pre.sh
ExecStart=/bin/my_program
RootDirectoryStartOnly=yes
```

Here *RootDirectory* shows where the root directory is for the *foobar* process.

 **Note:** The program script path has to be inside *chroot*, which makes the full path of that process script as */chroot/bin/my_program*.

Before the daemon is started, a shell script *pre.sh* is invoked, the purpose of which is to set up the *chroot* environment as necessary, i.e., mount */proc* and similar file systems into it, depending on what the service might need. You can start your service by using the following command:

```
#systemctl start my_chroot_Service.service
```

Ip-netns

The *Ip-netns* utility is one of the few that directly use network

namespaces to create virtual interfaces. To create a new network namespace, use the following command:

```
#ip netns add netns1
```

To check the interfaces inside, use the command shown below:

```
#ip netns exec netns ip addr
```

You can even get the shell inside it, as follows:

```
#ip netns exec netns /bin/bash
```

This will take you inside the network namespace, which has only a single network interface with no IP. So, you are not connected with the external network and also can't ping.

```
#ip netns exec netns ip link set dev lo up
```

This will bring the loop interface up. But to connect to the external network you need to create a virtual Ethernet and add it to *netns* as follows:

```
# ip link add veth0 type veth peer name veth1
# ip link set veth1 netns netns1
```

Now, it's time to set the IP to these devices, as follows:

```
# ip netns exec netns1 ifconfig veth1 10.1.1.1/24 up
# ifconfig veth0 10.1.1.2/24 up
```

Unshare

The *unshare* utility is used to create any namespace isolated environment and run a program or shell inside it.

To get a network namespace and run the shell inside it, use the command shown below:

```
#unshare --net /bin/bash
```

The shell you get back will come with a different network stack. You can check this by using *#ip addr*, as follows:

```
1: lo: <LOOPBACK> mtu 65536 qdisc noop state DOWN mode
DEFAULT group default
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
```

To create a user namespace environment, use the following command:

```
#unshare --user /bin/bash
```

You can check your user inside the shell by using the command below: